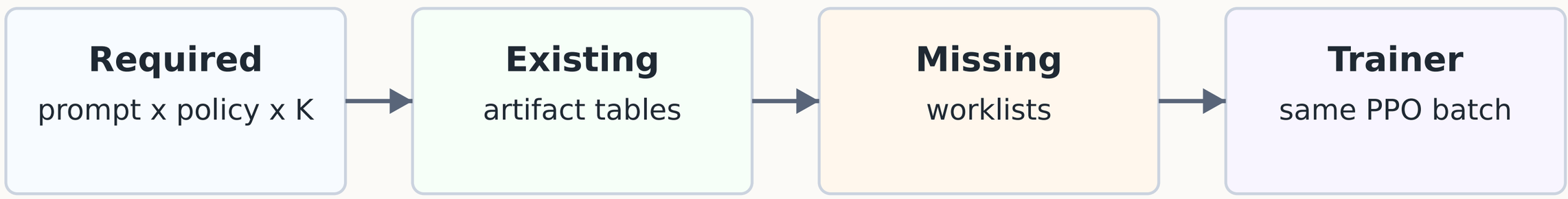
Target: minimize executor calls

while still producing the exact PPO batch relation

RelPT planning equation

required PPO rows - existing materialized rows = missing executor work

The optimizer target is small missing work, not a different PPO objective.

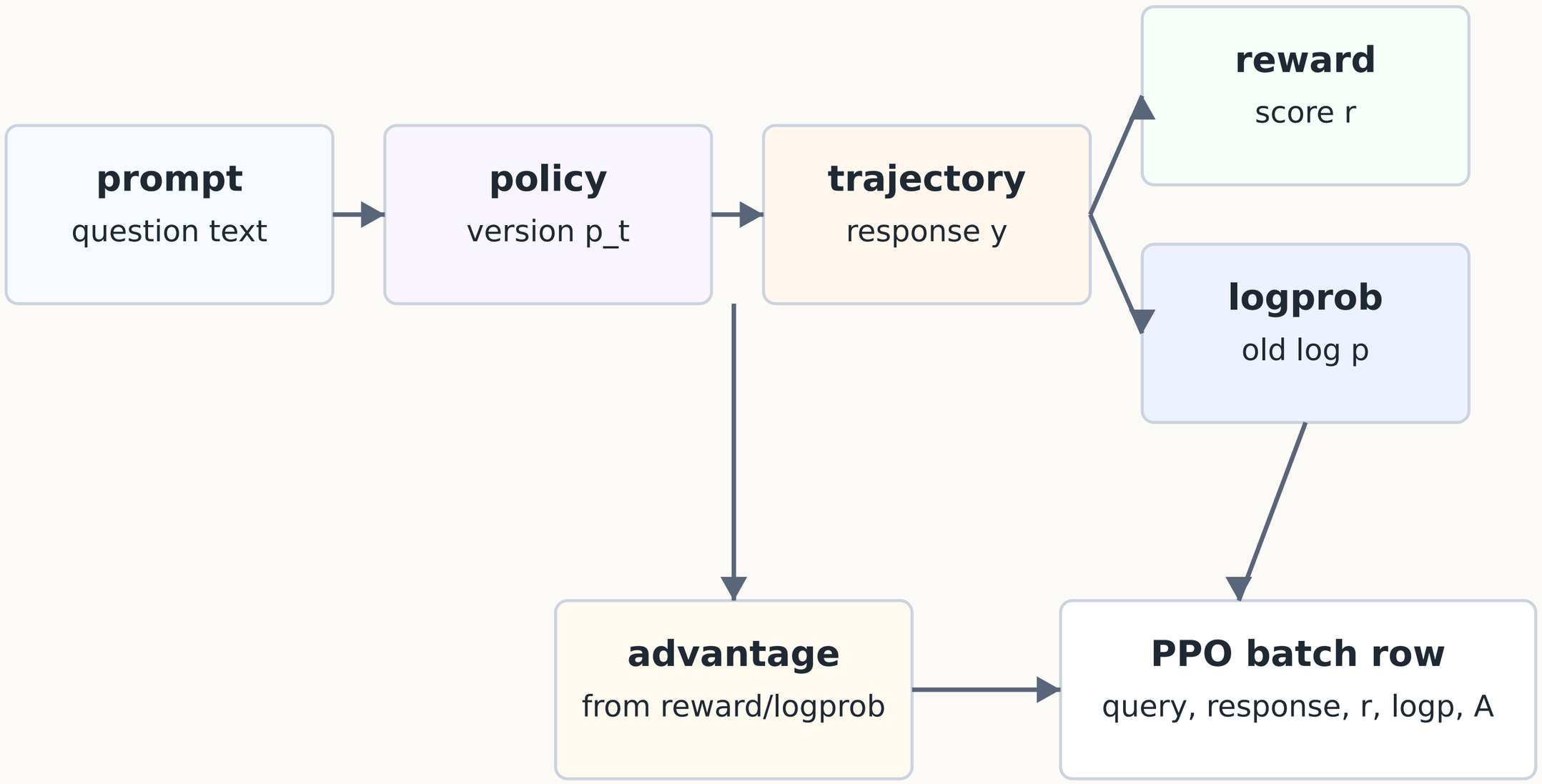


Concrete target example

For 96 prompts, suppose the next PPO batch asks for K=6 responses per prompt, but the database already has K=4 usable trajectories for the same policy. The required relation has 576 trajectory rows. The existing relation has 384. The missing relation has 192 rows. Only those 192 rows should call rollout.

A PPO batch is already a join of artifacts

One train row is valid only when all required artifacts exist.



Relational formulation

- trajectory joins prompt on prompt_id and policy on policy_id
- reward joins trajectory on traj_id plus reward_model_id
- logprob joins trajectory on traj_id plus old_policy_id
- advantage joins reward/logprob for derived training credit
- batch is the materialized set of complete joined rows

SQL JOIN is not cosmetic; it is the PPO data contract made explicit.

Anti-join planning finds missing rows

SQL asks: required rows LEFT JOIN existing rows, then keep NULL matches.

Repeated for every artifact layer

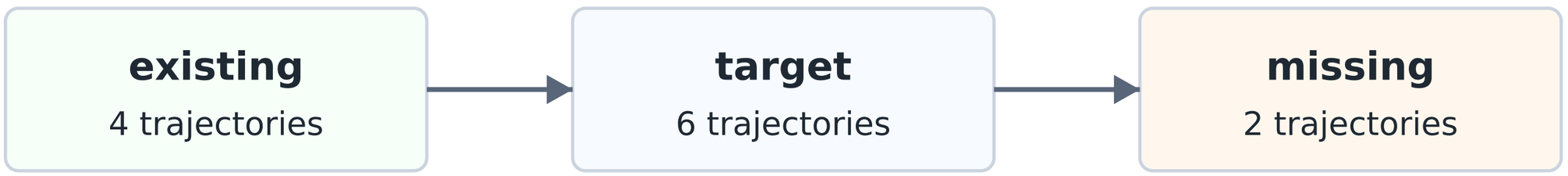
rollout	prompt x policy x K	-	trajectory	-> rollout_requests
reward	trajectory x reward_model	-	reward/cache	-> reward_traj_ids
logprob	trajectory x old_policy	-	logprob	-> logprob_traj_ids
advantage	rewarded trajectory	-	advantage	-> advantage_traj_ids
batch	complete joined rows	-	batch	-> materialized batch

DB role: physical joins, indexes, and materialization. RelPT supplies the artifact graph.

Example: partial rollout, K=4 to K=6

The target relation expands; existing rows stay valid.

Per prompt



Naive pipeline

96 prompts x 6

= 576 rollout calls

384 reusable rows are ignored

RelPT

96 prompts x (6 - 4)

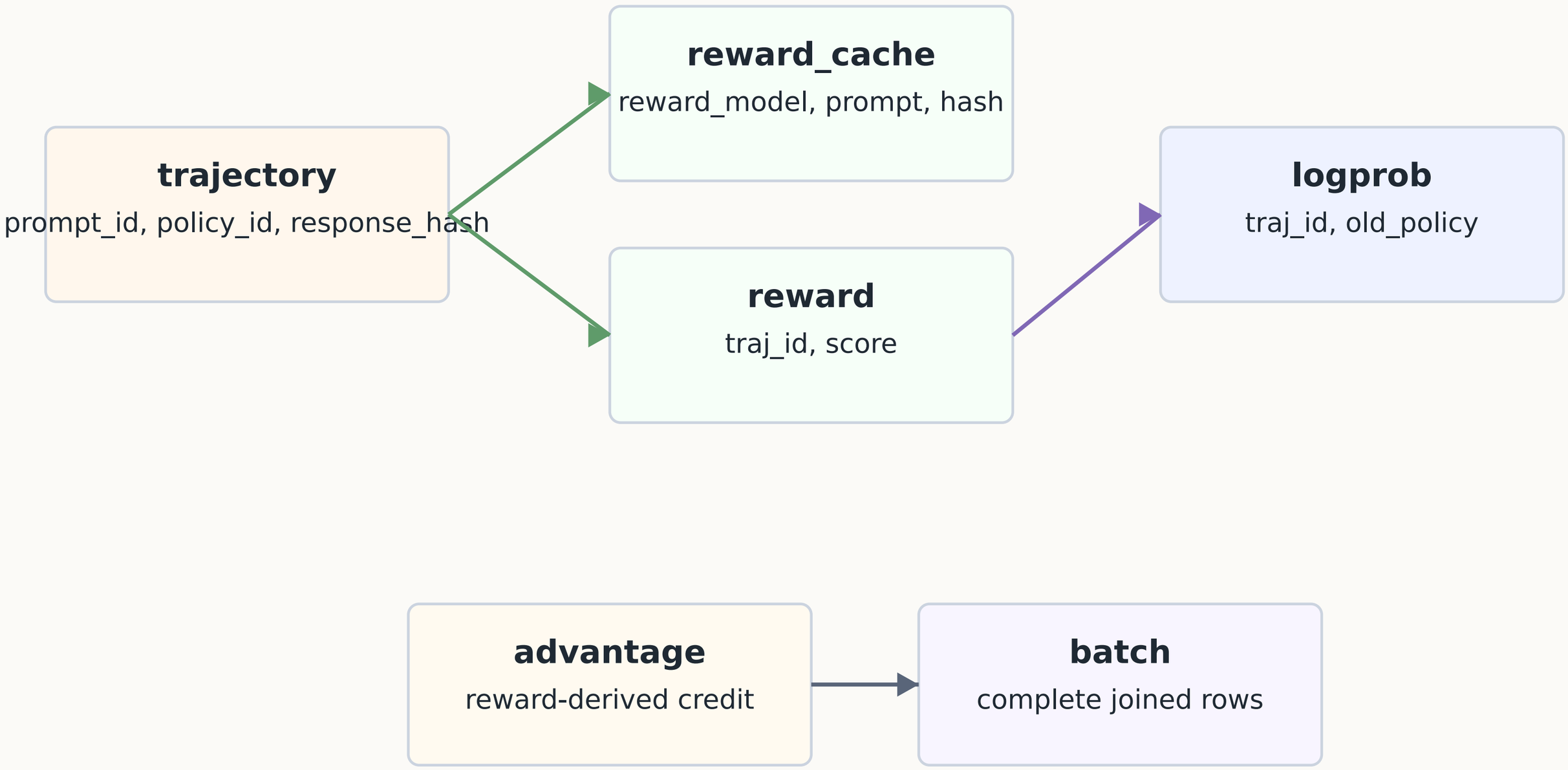
= 192 rollout calls

only missing rows call rollout

SQL sketch

```
GROUP BY prompt_id, policy_id; existing_k = count(trajectory)
rollout_requests = target_k - existing_k where existing_k < target_k
```

Rows unlock downstream joins as tables grow



Reuse rules are different for each table

`trajectory`	reused for same prompt, same policy, same rollout slot or response row
`reward_cache`	reused across policy versions if prompt + normalized response hash match
`logprob`	usually policy-specific; missing old-policy logprob must be computed
`advantage`	derived from reward/logprob; recompute only when inputs or baseline change
`batch`	materialized complete join; exact retry can reuse with zero executor calls

This is why the tables interact: a new upstream row changes which downstream joins are r

Why a DB optimizer can help

Stable keys prompt_id, policy_id, traj_id, reward_model_id, response_hash

Append-only facts executor outputs are durable rows, not hidden process state

Completeness query PPO input is complete only after the artifact joins succeed

Anti-join worklists missing rows are exactly what expensive executors must prod

Cost asymmetry rollout/reward/logprob are costly; SQL joins and indexes are ch

Important boundary RelPT optimizes artifact construction, not PPO loss math.

Measured effects: less repeated work

Same trainer path; savings come from artifact reuse.

Readout

TRL saved 49.3% prep time and 50.0% generated rows. The full SQLite prototype saved 99.3% reward executor calls because reward_cache reused deterministic scores.

What the example proves

Target

Produce the complete PPO batch relation with minimum additional executor calls.

Mechanism

Represent artifacts as growing tables; use joins and anti-joins to find missing rows.

Examples

Retry becomes zero new work. Partial rollout K=4 to K=6 becomes only the K delta. Deterministic rewards hit reward_cache.

Boundary

RelPT does not claim a better PPO loss. It claims a better systems plan for constructing PPO inputs.

Shortest conclusion: RelPT optimizes the dataflow before the gradi